

ControlPlane.ai

A real-time oversight layer for enterprise AI. It sits between an organisation's AI applications and the models they call, scores every single interaction on Performance, Cost and Responsibility, converts each finding into a dollar figure, and routes the decision through confidence-tiered human escalation instead of a binary block-or-allow.

 CI passing

 License MIT

 tests 70 passing

 python 3.12+

Built by **Team StratAI** — Krishna Gahlod, Mrunal Pachpande, Rudraksh Sharma (IIT Bombay) — for the **Accenture Innovation Challenge 2026**, Round 2, Problem Statement 1.

This is a working system, not a mockup. It makes genuine Gemini API calls, evaluates them end to end, and has been verified from a clean clone on a fresh machine.

Run it	\$7 Running it — two commands per service, no Docker required
What's real vs simplified	\$5 — stated plainly, including what we did <i>not</i> build
Why some checks are deterministic and others use an LLM	\$3
Measured results	\$4 — precision, recall and latency, each with the command that produced it
Business proposal, market research, audit	docs/

Contents

1. The problem
2. Architecture
3. Detection: deterministic vs LLM-as-judge
4. Measured results
5. What's real, what's simplified — and why
6. Tech stack and design system
7. Running it
8. Guided walkthrough
9. API reference
10. Repository structure
11. Known limitations and roadmap
12. Documentation index

1. The problem

Enterprises run generative AI across many use cases at once — a customer-facing support bot, an internal knowledge copilot, a regulated decision-support tool — each with a different risk tolerance, latency budget and failure cost. The failures are only discovered *after* a customer acts on a hallucination, a regulator asks a question, or the invoice arrives.

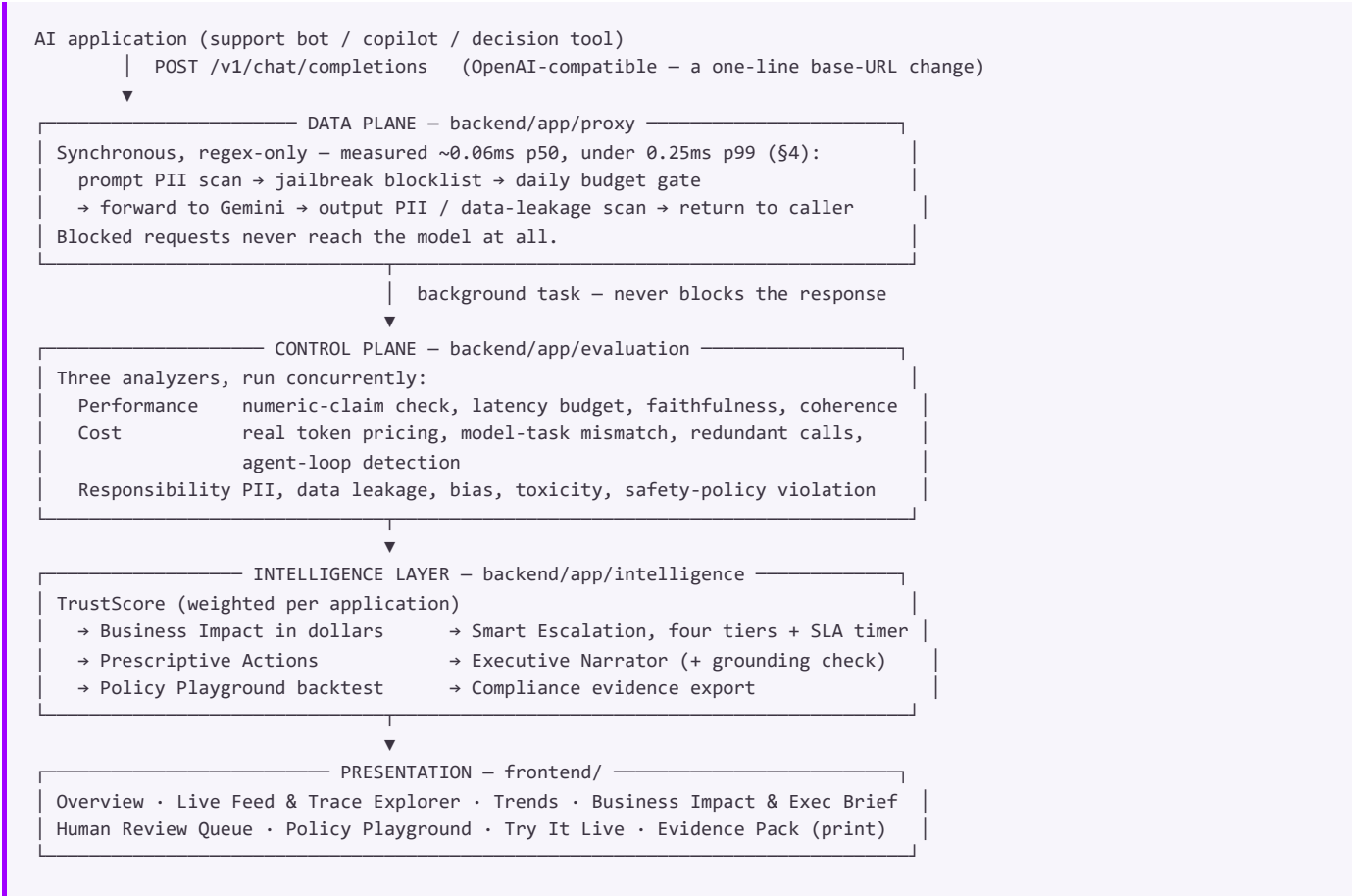
Three risk categories matter, and they **overlap**, which is why single-category tooling misses them:

RISK	WHAT GOES WRONG
Performance	The model is confidently wrong — cites a figure that appears nowhere in its source, or contradicts the document it was given.
Cost	Agents loop without converging; frontier models answer trivial questions. The spend is invisible until the bill arrives.
Responsibility	PII leaks, bias, prompt injection — undetected until they become a compliance incident.

A response that invents a customer's contact details is **a faithfulness failure and a privacy incident at the same time**. ControlPlane.ai tags flags by dimension rather than forcing each into one bucket, and scores all three dimensions concurrently.

2. Architecture

Four layers. Only the first sits in the user's path.



The scoring model

TrustScore is a 0–100 weighted composite of the three dimension scores. The weights are **per application**, not global — that is the point:

APPLICATION	TYPE	PERFORMANCE	COST	RESPONSIBILITY	LATENCY BUDGET
Customer Support Bot	Customer-facing	40%	25%	35%	4s
Internal Knowledge Copilot	Internal	35%	40%	25%	8s
Underwriting Decision-Support	Regulated	30%	15%	55%	12s

Each application also carries its own **system prompt** — its operating instruction, persona and refusal rules — stored as configuration, not code. A caller-supplied system message layers *on top* of it rather than replacing it, so an application's guardrails cannot be removed by the request they exist to constrain.

The four escalation tiers

TrustScore drives a graduated response, not a binary decision:

TRUSTSCORE	DECISION	BEHAVIOUR
90–100	<code>allow_silent</code>	Delivered. No human ever sees it. The majority of traffic.
70–89	<code>allow_flag_async</code>	Delivered, recorded for asynchronous review and trend analysis.

TRUSTSCORE	DECISION	BEHAVIOUR
30–69	<code>escalate_human</code>	Enters the Review Queue with full context and an SLA countdown. Approve, reject, or edit and approve .
0–29	<code>auto_block_alert</code>	Blocked, alert raised.

Two override rules sit on top: a **critical safety violation always forces a block** regardless of score, and **PII that was already auto-redacted does not additionally force an escalation** if nothing else is wrong. When an SLA expires, a safe default applies automatically and is recorded as `auto_defaulted` — a queue backlog can never become a silent failure.

3. Detection: deterministic where it must be, LLM where it has to be

The Round 2 brief asks teams to justify when logic is deterministic and when it is model-based. We treat that as a credibility test, and **every flag in the product is labelled with the method that produced it** — visible in the UI, the API and the compliance export.

CHECK	METHOD	WHY THIS METHOD
Input/output PII, jailbreak blocklist, data leakage (API keys, internal URLs)	Deterministic — regex and entity match	Must run on the synchronous path. Measured well under 1ms at p99 (\$4) — sub-millisecond in fact, not just "fast for an LLM call".
Numeric-claim hallucination — a figure in the answer absent from the source	Deterministic — number extraction and set comparison	A quantitative claim must never depend on a model's assessment of itself.
Token cost, pricing, model–task mismatch	Deterministic — real token counts × a pricing table, keyword complexity heuristic	Cost arithmetic must never be LLM-generated.
Response latency against the application's budget	Deterministic — measured against each app's own budget	A stopwatch comparison should never be delegated to a model.
Faithfulness, contradiction, completeness, coherence	LLM-as-judge (Gemini)	No automated ground truth exists for open-ended claims. Labelled a judgement, never a measurement.
Bias, toxicity, safety-policy violation	LLM-as-judge (Gemini)	Requires semantic understanding no regex can provide.
Business impact in dollars	Deterministic formula over published assumptions	A rule-based mapping table, not a model inventing a number.
Executive Narrator prose	LLM-generated , then deterministically grounding-checked	The numbers are never invented; only the sentences wrapping them are. See below.

On the current seed dataset that split is **64 deterministic, 30 rule-based, 63 LLM-as-judge** findings — all individually labelled. **94 of 157 (60%) are reached without an LLM**: the ones that survive a quota outage, cost nothing per call, and return the same verdict every time. Counted by `python -m eval.report` , not by hand.

Where no source context is supplied, an LLM-as-judge finding is tagged **unverifiable** and reported at reduced confidence. The system states that it could not verify, rather than implying that it did.

We govern our own AI feature too

The Executive Narrator is itself an LLM feature, so it is held to the standard we hold the applications we monitor to. During development it **hallucinated** — inventing two service names and a remediation story that existed nowhere in our system. Our own product committing the exact failure it exists to catch. The fix was not just a better prompt:

1. The real monitored application names are passed into the prompt, so the model has true nouns to reference instead of manufacturing them.
2. A **deterministic grounding check** (`backend/app/intelligence/grounding.py`) extracts every named entity from the generated narrative and verifies it is reconstructible from the source statistics. No second model is involved — a grounding guarantee that itself depends on a probabilistic model is not a guarantee.
3. On failure it regenerates once, naming the fabrications. If that also fails, a deterministic template built directly from the numbers is served instead.
4. The verdict is **displayed on screen**, not hidden.

An ungrounded claim cannot reach the user. Worst case, the reader sees a plainer report that is provably true.

4. Measured results

Claims about a detector are worth what their measurements are worth, so every figure below is generated by a command in this repository and written to a committed file. Re-run the command and you either reproduce the file or see a diff.

```
cd backend
python -m eval.report          # → reports/evaluation.md
python -m eval.bench_latency   # → reports/latency.md
```

The corpus is **165 labelled interactions — 96 carrying a real problem, 69 clean**. Each label was written when the scenario was authored, before any analyzer ran against it, so the labels are independent of the scores used to judge them.

Detection — did we notice?

MEASURE	VALUE	
Recall	1.000	every one of the 96 problems produced a finding
Precision	0.881	96 of 109 findings were on real problems
False-positive rate	0.188	13 of 69 clean interactions also drew a finding

Detection is deliberately noisy. A finding is cheap — it is recorded and it moves a trend line, and it costs nobody an interruption.

All 13 false positives come from a single detector, `redundant_call` , which fires on a Jaccard similarity above 0.85 between prompts in a recent-time window — legitimately distinct FAQ questions share most of their wording and

clear that bar. No PII, safety, bias or grounding check fired on a single clean interaction; precision on those is 1.000 on this corpus. That is a bounded problem in one threshold rather than diffuse inaccuracy, and it is the first thing worth tightening.

Routing — did we act?

TIER BOUNDARY	FIRES ON	PRECISION	RECALL
Any routing change (< 90)	37	1.000	0.385
Human attention (< 70)	9	1.000	0.094
Automatic block (< 30)	0 from score alone	—	—

No clean interaction crossed any tier boundary. The lowest-scoring clean interaction scored 95.2, leaving a 5.2-point margin above the first boundary across 69 clean samples.

A published miss

Detection recall is 1.000. Routing recall is 0.385. **59 of 96 problems were detected and recorded but scored high enough that nothing about their handling changed.**

The cause is structural rather than a detector weakness: TrustScore is a weighted average, and a response that is factually contradictory but cheap and perfectly safe fails one dimension while passing two, so the mean lands above 90. The fix is not more sensitivity — lowering the boundary would spend that 5.2-point margin and start catching clean traffic. It is to route on *consequence*: the estimated business impact and reversibility that the system already computes and currently ignores at decision time. That is on the roadmap in §11, and the measurement is published now because it is what the corpus shows.

Why the system grades instead of blocking

The corpus is 58% problems by construction. Production traffic is not, and precision depends on the base rate however good a detector is. Applying Bayes' rule to the *measured* rates above:

	AT 1% PREVALENCE	AT 5%	AT 10%	ON THIS CORPUS (58%)
Detection layer	5.1%	21.8%	37.1%	88.1%
Routing at < 90 (worst case)	8.2%	31.8%	49.6%	100%

At a 1% base rate, a detector with perfect recall and our measured false-positive rate would be right about **5% of the time** — roughly 19 false alarms in every 20. That is arithmetic, not a defect, and no amount of tuning removes it.

This is the argument for the whole design: at realistic prevalence no automated tier is precise enough to be trusted on its own. Which is why the tiers do different things rather than the same thing at different sensitivities — a flag is allowed to be noisy because it costs nothing, an escalation puts a human in front of the evidence, and a block is reserved for the override path where the evidence is categorical rather than statistical.

The routing row is deliberately pessimistic: routing produced *zero* false positives across 69 clean samples, which would project to 100% precision at every prevalence. Zero observed is not zero true, so the table uses the rule of three — the 95% upper bound after 69 clean trials with no events, $FPR \leq 0.043$.

Control-plane overhead

Only code in this repository is timed. How long Gemini takes is a property of the provider, not something an oversight layer chooses, so it is never folded into a claim about our own cost.

PATH	P50	P95	P99	TARGET
check_prompt	0.018 ms	0.035 ms	0.053 ms	< 10 ms — met
check_response	0.031 ms	0.072 ms	0.096 ms	< 10 ms — met
Data Plane, full request	0.057 ms	0.141 ms	0.207 ms	< 10 ms — met
TrustScore + routing decision	0.002 ms	0.002 ms	0.003 ms	< 1 ms — met

Single-process, single-machine figures for the cost of the code itself, not a load test; under real concurrency the tail would widen. **These move between runs** — at this scale the measurement is dominated by scheduler noise, so prose elsewhere quotes a stable bound (p50 around 0.06 ms, p99 comfortably under 0.25 ms) rather than a figure the next run would contradict. Full caveats in `reports/latency.md`.

5. What's real, what's simplified — and why

Everything in §2 and §3 is implemented and running. The following simplifications were made deliberately, and are documented rather than hidden.

Deliberate infrastructure choices

SIMPLIFICATION	WHY
SQLite, not Postgres	Zero setup for a reviewer running this locally.
In-process state, not Redis/Celery	Budget counters and the redundant-call cache are in-memory; async dispatch uses FastAPI <code>BackgroundTasks</code> + <code>asyncio.to_thread</code> . A real queue is the first roadmap item.
No Presidio/Detoxify model downloads	PII detection is a curated regex module; bias and toxicity use LLM-as-judge. A direct tradeoff against multi-gigabyte downloads.
Compressed SLA windows (120s standard, 30s critical)	So the safe-default behaviour is observable live instead of requiring a five-minute wait.
Deterministic seed data	165 pre-authored interactions across 53 distinct prompts, with hand-labelled ground truth, written through the <i>real</i> scoring, impact and escalation code paths. Gives reproducible history without burning API quota.
No multi-tenant auth	Single-process demo. Noted in §10.

Stated business assumptions

Every dollar figure traces to `backend/app/config.py: BUSINESS_ASSUMPTIONS`. Nothing is a magic number embedded in prose:

ASSUMPTION	VALUE
Average order value	\$85
Customer lifetime value	\$620
Weekly interactions per application	12,000
GDPR fine reference	\$20,000,000
Probability of a fine per PII incident	0.04%
Remediation cost per compliance incident	\$1,500
Reputation incident base cost	\$8,000

Worked example: a single auto-redacted PII leak computes to **(\$20M × 0.04%) + \$1,500 ≈ \$9,500** of expected exposure — traceable line by line.

On the dashboard's cumulative total. The seeded demonstration surfaces roughly **\$740K** of estimated exposure. That figure is **demo-representative, not production-representative**: the seed mix over-indexes on failure cases by design. The Business Impact Scorer extrapolates each flag across the application's full traffic volume, so multiplying a per-flag estimate by a count of flagged interactions would double-count the same exposure. We would rather name that than make it.

Working with free-tier model quotas

Free-tier Gemini quota is **per model, per project, per day**, so an exhausted model is a routine condition rather than an outage. The system handles it in three places:

- Every logical call **walks a configurable ladder of models** (`GEMINI_FALLBACK_MODELS`) before giving up, so one exhausted tier does not take the pipeline down.
- Each LLM-as-judge call **degrades to "no signal from this check"** rather than crashing the evaluation. This is a **fail-open**, and it is worth naming as one: when every judge tier is exhausted the evaluation completes with the deterministic findings only, so the interaction can score well because a check *did not run* rather than because it passed. The exhaustion is logged, but it is not currently recorded on the interaction itself. See §11.
- The Executive Narrator falls back to its deterministic template, so a quota exhaustion produces a plainer *real* report rather than an error message.

Provider errors are also mapped to a short, actionable sentence rather than surfacing the raw JSON — the difference between "quota exhausted, retry in 30s" and a stack trace on screen.

6. Tech stack and design system

- **Backend** — Python 3.13, FastAPI, SQLAlchemy 2.0, SQLite, `google-genai` SDK.
- **Frontend** — Next.js 16 (App Router), TypeScript, Tailwind CSS 4, Recharts, Framer Motion.
- **No Docker requirement** — plain `uvicorn` and `npm` .

Design system

The UI follows an Accenture-style enterprise visual language: light theme, the brand purple `#A100FF` (with `#7500C0` as the text-safe deep variant) as the sole accent, and **Public Sans** as the UI typeface — the closest freely licensable alternative to Graphik, which is a paid commercial font. The palette was **extracted from the theme XML of a real Accenture-branded deck**, not guessed.

Chart colours are validated, not chosen by eye. The categorical series palette was checked with an accessibility validator for colour-vision-deficient adjacent-pair separation and contrast against a white surface. Status and severity colours are kept deliberately distinct from the series palette so a status colour never impersonates a data series, and the Trends page uses two single-axis charts rather than one dual-axis chart.

Model output is rendered through a small hand-written formatter rather than a markdown library — this is untrusted model text, and building React elements from a known-small grammar avoids `dangerouslySetInnerHTML` entirely.

7. Running it

Prerequisites: Python 3.11+, Node.js 20+, and a Gemini API key (aistudio.google.com/apikey — the free tier is sufficient).

Backend

```
cd backend
python -m venv venv
./venv/Scripts/activate          # Windows.  macOS/Linux: source venv/bin/activate
pip install -r requirements.txt

cp .env.example .env
# edit .env and set GEMINI_API_KEY=<your key>

python -m app.seed.seed_data      # seeds 165 historical interactions across 3 applications
uvicorn app.main:app --reload --port 8000
```

The API is now live at `http://localhost:8000`. Verify it end to end:

```
curl -X POST http://localhost:8000/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{"messages":[{"role":"user","content":"What is your return policy?"}],
  "metadata":{"app_key":"customer_support_bot"},
  "rag_context":"Returns are accepted within 30 days with a valid receipt."}'
```

Changing the API key later requires a backend restart. Settings are cached and the model client is constructed at import time, so an edit to `.env` has no effect until the process restarts. Note also that the file the backend reads is `backend/.env` — a `.env` at the repository root is not read by anything.

Frontend

```
cd frontend
npm install
cp .env.example .env.local      # defaults to http://localhost:8000
npm run dev                     # http://localhost:3010
```

For a demo or recording, run a production build instead — the dev server renders an error-count badge in the corner whenever any console error occurs:

```
npm run build && npm run start
```

Demo mode (optional)

The Review Queue's SLA windows are deliberately short, which is correct product behaviour and awkward to demonstrate — an item can expire mid-walkthrough. Setting `DEMO_MODE=1` in `backend/.env` enables:

ENDPOINT	PURPOSE
<code>GET /api/demo/status</code>	Reports whether demo mode is on
<code>POST /api/demo/arm-review-queue</code>	Re-opens three recent human-review escalations — chosen across different applications and decision types — with a 10-minute SLA

It re-opens escalations that **genuinely reached** a human-review decision; it does not fabricate interactions. Off by default, and documented here rather than hidden, because an AI-governance tool should not ship an undisclosed way to manufacture its own state.

8. Guided walkthrough

#	PAGE	WHAT TO LOOK FOR
1	Overview	Fleet-wide TrustScore, estimated exposure in dollars, and per-application budget usage. Each application is governed by different weights.
2	Live Feed	Turn on Flagged only . Open the <code>redacted</code> Customer Support Bot trace about a double-charged order: the model invented a refund amount <i>and</i> a customer's contact details. The invented identity was redacted synchronously; the invented number was caught asynchronously. Both detection methods appear side by side.
3	Trends	TrustScore and its three sub-dimensions over 14 days. A drop in Responsibility and a drop in Cost are different problems with different owners.
4	Business Impact	The same data narrated for Engineer, CISO and CEO — each with the green grounding-verified badge beneath it.
5	Review Queue	Pending escalations with live SLA countdowns and a three-way action: approve, reject, or edit and approve .

#	PAGE	WHAT TO LOOK FOR
6	Policy Playground	Drag the threshold and watch precision, recall, F1 and false-positive rate recompute against 165 labelled interactions. At the recommended threshold of 95: 76 blocked, precision 1.00, recall 0.792, FPR 0.0%, F1 0.884.
7	Try It Live	Send a real request through the proxy. Watch the six-stage pipeline advance. The Jailbreak preset halts at the synchronous guardrail with the model stage marked <i>never called</i> .
8	Evidence pack	From any trace, open the compliance evidence pack — prompt, both response versions, every finding with its detection method, the impact estimate with its assumptions, and the governance decision. Prints straight to PDF.

9. API reference

All responses are JSON unless noted.

Proxy (Data Plane)

METHOD	PATH	DESCRIPTION
POST	/v1/chat/completions	OpenAI-compatible. Accepts <code>messages</code> , optional <code>rag_context</code> , and <code>metadata.app_key</code> . Returns the model response plus a <code>controlplane</code> envelope with the interaction id, sync action, sync flags, latency and whether the model was called.

Dashboard

METHOD	PATH	DESCRIPTION
GET	/api/apps	Monitored applications with policy weights and budget usage
GET	/api/summary?days=	Fleet-level headline statistics
GET	/api/interactions?app_id=&limit=	Interaction feed
GET	/api/interactions/{id}	Full trace including raw vs delivered response
GET	/api/trends?app_id=&days=	Daily TrustScore, sub-scores, volume and spend
GET	/api/alerts · /api/recommendations	Deduplicated alerts; prescriptive actions
GET	/api/impact-breakdown?days=	Estimated impact by risk category

Intelligence

METHOD	PATH	DESCRIPTION
GET	/api/narrator?audience=&days=	Persona narrative plus its grounding verdict
GET	/api/playground/simulate?threshold=	Precision, recall, F1, FPR and confusion matrix at a threshold

METHOD	PATH	DESCRIPTION
GET	/api/playground/recommend	Recommended threshold with the reasoning
GET	/api/review-queue?status=	Escalations awaiting or past decision
POST	/api/review-queue/{id}/decision	approve , reject or edit , with an optional note

Compliance

METHOD	PATH	DESCRIPTION
GET	/api/interactions/{id}/evidence	Structured evidence pack for one interaction
GET	/api/export/flagged.csv	One row per finding across the period, with detection method

10. Repository structure

controlplane-ai/	
├─ backend/	
│ └─ app/	
│ ├── proxy/	Data Plane – OpenAI-compatible endpoint, sync checks, Gemini client with the model fallback ladder
│ ├── evaluation/	Control Plane – performance / cost / responsibility analyzers, scoring, orchestrator
│ ├── intelligence/	TrustScore, business impact, escalation, prescriptive actions, executive narrator, grounding check, playground
│ ├── api/	REST routes – dashboard, review queue, playground, narrator, compliance export, demo mode
│ ├── db/	SQLAlchemy models
│ ├── seed/	Application definitions and the deterministic seeder
│ └─ config.py	Policy tiers, pricing, business assumptions, patterns
├─ eval/	Evaluation harness – writes the committed reports/
└─ tests/	70 pytest cases over scoring, routing, detection, grounding
├─ frontend/	
│ ├── app/	One route per dashboard view, plus the print evidence pack
│ ├── components/	UI primitives, pipeline visualisation, model-output renderer
│ └─ lib/	API client, connection state, formatting, types
├─ reports/	Committed, reproducible evidence – not build output
├─ docs/	Architecture, market research, sources, audit, demo script
├─ .github/workflows/	CI – backend tests and a frontend production build
├─ BUSINESS_PROPOSAL.md	Full Round 2 business proposal
├─ LICENSE	MIT
└─ README.md	

11. Known limitations and roadmap

Named deliberately. A proposal claiming to have solved these in a hackathon prototype would not survive technical due diligence.

Current limitations

- **Score-based routing under-reaches, and we measured it.** Detection recall is 1.000, but routing recall at the first tier boundary is 0.385 — 59 of 96 labelled problems were detected and recorded without their handling changing (§4). TrustScore is a weighted average, so a single-dimension failure is diluted by two passing dimensions. This is the most significant known gap in the system.
- **Judge unavailability fails open and is not recorded on the interaction.** If every model tier is exhausted, the semantic checks silently contribute nothing and the trace does not distinguish "this check found nothing" from "this check never ran". The deterministic 60% of findings are unaffected, which bounds the blast radius, but an operator cannot currently see the difference. The right fix is to treat a detector failure as an operational event that is never a content signal, and to make the response to one a per-application policy — fail-open for a support bot, fail-closed to human review for a regulated workflow.
- **No multi-tenant authentication** on the proxy endpoint — single-process demo.
- **In-process state** rather than a real message queue; this is the binding constraint on evaluation throughput.
- **No multi-turn conversation-state tracking.** The Cost Analyzer's agent-loop detector watches for repeated non-converging calls, but full conversational risk accumulation is not implemented.
- **The feedback loop captures but does not yet close.** Every reviewer decision is persisted against the original flags, ready for threshold recalibration — the mechanism is built, the automatic recalibration is not.
- **LLM-as-judge latency and cost** are acceptable at demo volume; at production volume a purpose-built classifier is the right answer. The analyzer interfaces do not assume Gemini.

Roadmap

PHASE	SCOPE
Phase 1 — 2–3 months	Consequence-aware routing — escalate on expected loss (estimated business impact × probability of error) and on reversibility, both already computed and currently unused at decision time; this is the direct answer to the §4 routing gap. Redis Streams message queue; purpose-built classifiers to reduce judge dependency; a policy-as-code editor; automated threshold recalibration from captured reviewer decisions.
Phase 2 — 3–6 months	Per-application <code>fail_mode</code> — fail-open or fail-closed on detector failure, recorded as an operational event distinct from any content finding. Design-partner deployment on a non-critical use case; multi-turn conversation-state tracking; ISO 42001 / EU AI Act evidence-bundle export.
Phase 3	Multi-tenant auth and SOC 2 readiness; horizontal scaling of the evaluation layer; industry policy-template packs.

12. Documentation index

DOCUMENT	WHAT IT COVERS
<code>StratAI_ControlPlane.ai_Business-Proposal.pptx</code>	The submitted business-proposal deck, 24 slides
<code>ControlPlane.ai_README.pdf</code>	This document as a PDF, for the submission portal
<code>BUSINESS_PROPOSAL.md</code>	The full Round 2 business proposal — market, differentiation, economics, go-to-market, risks
<code>docs/market_research_report.md</code>	Market sizing, competitor funding and regulatory timeline, with 40+ cited sources

DOCUMENT	WHAT IT COVERS
docs/architecture_deep_dive.md	The full architecture design this prototype implements
docs/product_audit_report.md	A hands-on UX and engineering audit of the running product, with severity-ranked findings
docs/implementation_plan.md	How those findings were resolved, and what was found during the work
docs/demo_video_script.md	Scene-by-scene demo script with a pre-flight runbook
docs/competitor_analysis.md	Round 1 competitive analysis, re-verified in Round 2
docs/sources.md	Every figure we quote, with its source, date and confidence — and which numbers are our own assumptions rather than findings
reports/evaluation.md	Generated: detection and routing quality against the labelled corpus
reports/latency.md	Generated: control-plane overhead, measured

Team StratAI · IIT Bombay · Accenture Innovation Challenge 2026